DISEÑO Y DESARROLLO DE JUEGOS INTERACTIVOS 2

MANUAL DE DESARROLLO DEL VIDEOJUEGO

MEGAFIGHTERS

Combate de plataformas, armas, enemigos y progresión por niveles

Motor: Godot Engine 4.7 (configuración del proyecto)
Genero: acción y plataformas 2D
Estructura: menú principal + 3 niveles + escena final

Propósito
Documentar las escenas y mecánicas presentes en Megafighters, explicar cómo abrirlo y jugarlo, y ofrecer una ruta de revisión y ampliación paso a paso.

La guía adjunta describe otro juego. Se toma como referencia su organización paso a paso; sus instrucciones específicas no se consideran requisitos de Megafighters. El contenido de este manual se basa en las escenas y scripts del proyecto.

Contenido

1. Presentación y propósito
2. Requisitos y puesta en marcha
3. Estructura del proyecto
4. Configuración y controles
5. Menú principal y flujo de juego
6. Movimiento del personaje
7. Combate y sistema de vida
8. Armas y aparición en el escenario
9. Enemigos y progresión
10. Plataformas, escaleras y portales
11. HUD e interfaz
12. Arquitectura y patrones
13. Reinicio, victoria y continuidad
14. Niveles y escena final
15. Audio y recursos visuales
16. Pruebas recomendadas
17. Errores frecuentes
18. Actividades de aprendizaje
19. Lista de verificación
20. Referencias y créditos
21. Próximas mejoras

Resumen: atraviesa escenarios de plataformas, derrota a todos los enemigos de cada nivel y avanza. Puedes correr, saltar, agacharte, hacer roll dive, atacar y recoger armas. Escaleras, plataformas móviles y portales amplían las rutas.

1. Presentación y propósito del videojuego

Megafighters es un juego de acción y plataformas 2D creado en Godot. Combina desplazamiento lateral, combate, exploración vertical y objetos interactivos. Cada nivel administra una oleada de enemigos; cuando todos son derrotados, se muestra victoria y se puede continuar.

1.1 ¿Qué aprenderás?

- Organizar un juego en escenas reutilizables de Godot.
- Separar entrada, movimiento, combate, navegación y ciclo de vida en controladores.
- Conectar señales entre jugador, interfaz y administrador de enemigos.
- Crear apariciones de armas y enemigos desde puntos Marker2D.
- Aplicar Factory y Strategy para desacoplar creación y selección.
- Reconocer qué parte del proyecto sigue en desarrollo.

1.2 Reglas actuales

Elemento	Comportamiento en Megafighters
Objetivo	Derrotar la oleada de enemigos del nivel.
Jugador	Tiene puntos de vida y recibe daño, empuje y efectos de portal.
Armas	Se generan desde puntos configurados; hay armas cuerpo a cuerpo y de fuego.
Avance	Nivel 1 → 2 → 3 → escena final de rescate.
Derrota	Se muestra un mensaje; R permite reiniciar cuando está visible.

La pantalla final aún muestra “Escena de rescate por editar”. Se documenta como contenido pendiente, no como una secuencia terminada.

2. Requisitos y puesta en marcha

2.1 Requisitos

- Godot Engine 4.7 o compatible; project.godot declara Godot 4.7 y Forward Plus.
- Carpeta completa del proyecto con project.godot, scenes, scripts, assets y addons.
- Teclado para jugar.

2.2 Abrir y ejecutar

1. En Godot Project Manager, selecciona Importar.
2. Elige project.godot dentro de la carpeta Megafighters.
3. Espera a que Godot importe los recursos.
4. Ejecuta con F5 la escena principal o F6 la escena abierta.
5. En el menú principal, selecciona Jugar.

La escena principal definida corresponde a scenes/menu_principal.tscn. Si faltan recursos, verifica que abriste la carpeta raíz correcta y que se conservaron las rutas res://.

2.3 Complementos y configuración

El proyecto usa Godot 4, Jolt Physics y el complemento Live Palette. MusicManager está registrado como autoload. El complemento se encuentra en addons/live_palette; si la paleta no aparece en el editor, revisa Project Settings > Plugins.

3. Estructura del proyecto

```
res://
|-- project.godot
|-- scenes/
|   |-- menu_principal.tscn
|   |-- nivel1.tscn / nivel2.tscn / nivel3.tscn
|   |-- player.tscn / enemy.tscn
|   |-- plataforma_movil.tscn / escalera.tscn / Portal.tscn
|   |-- armas/ (10 escenas)
|   |-- hub.tscn / escena_final_rescate.tscn
|-- scripts/
|   |-- player.gd / enemy.gd / juego.gd / enemy_spawner.gd
|   |-- components/ / factories/ / strategies/ / iu/
|-- assets/ (sprites y audio)
|-- resources/ / addons/ / fonts/
```

3.1 Escenas y responsabilidades

Escena / archivo	Responsabilidad
menu_principal.tscn	Menú con botones Jugar y Salir.
nivel1/2/3.tscn	Escenario, jugador, enemigos, spawners, armas y HUD.
player.tscn + player.gd	Personaje controlado, estados, movimiento y combate.
enemy.tscn + enemy.gd	Enemigo con navegación, percepción, combate y ciclo de vida.
juego.gd	Coordina mensajes, derrota, reinicio y avance.
hub.tscn + hub.gd	Barras de vida del jugador y los enemigos.
escena_final_rescate.tscn	Marcador de posición que falta completar.

4. Configuración y controles

4.1 Controles

Acción	Tecla(s)	Resultado
Mover	A/D o flechas	Desplazamiento horizontal.
Saltar / subir	Espacio, W o arriba	Salto en suelo; subir en escalera.
Bajar / agacharse	S o abajo	Baja escalera y mantiene agachado.
Correr	Z	Aumenta velocidad.
Atacar	X	Ataque cuerpo a cuerpo.
Roll dive	Abajo + salto	Impulso; usa la orientación si no hay dirección.
Reiniciar	R	Reinicia tras mostrar derrota.
Continuar	Enter	Avanza tras mostrar victoria.

Varias teclas se leen directamente con Input.is_key_pressed; las flechas también usan acciones UI de Godot. attack (X) y reset (R) se declaran en project.godot.

4.2 Flujo de juego

1. Desde el menú, pulsa Jugar.
2. Explora las plataformas y escaleras; ataca a los enemigos.
3. Recoge armas y aprovecha los elementos del escenario.
4. Derrota toda la oleada; pulsa Enter si no hay transición automática.
5. Si el jugador muere, pulsa R cuando aparezca el mensaje.

5. Menú principal y flujo de juego

menu_principal.tscn muestra el título Megafighters y los botones Jugar y Salir. Jugar se conecta a scenes/iu/boton_jugar.gd; Salir usa scripts/iu/boton_salir.gd.

5.1 Coordinación de un nivel

Cada nivel incluye Juego, jugador y EnemySpawner. Juego crea una vista de mensajes y un GameFlowController. Este escucha la señal de enemigos derrotados, muestra victoria, permite continuar y coordina el reinicio.

```
EnemySpawner -> todos_los_enemigos_derrotados
Juego -> GameFlowController.enemigos_derrotados()
Vista de mensaje -> mostrar_victoria()
Enter -> carga escena_siguiente
```

Juego exporta espera_transicion_victoria. Si es mayor que cero, programa el cambio de escena; si es cero, el jugador continúa con Enter.

5.2 Comunicación por señales

enemy_spawned y enemy_removed actualizan las barras de vida del HUD. todos_los_enemigos_derrotados inicia la victoria. Las señales del jugador comunican muerte y solicitud de reinicio.

6. Movimiento del personaje

El jugador usa CharacterBody2D. scripts/player.gd coordina controladores especializados para evitar concentrar toda la lógica en una rutina.

Modo	Velocidad ref.	Activación
Agachado	120	Abajo / S.
Trote	240	Dirección normal.
Carrera	380	Z + dirección.
Escalera	190	Arriba o abajo en la escalera.
Roll dive	360 horiz.	Abajo + salto; animación o máximo 0,6 s.

6.1 Salto y plataformas

La fuerza de salto configurada es -520 y la gravedad base es 1400. Al dejar una plataforma móvil, el jugador conserva su velocidad según PLATFORM_ON_LEAVE_ADD_VELOCITY.

6.2 Controladores

Script	Tarea
PlayerInputController	Traduce teclas a acciones.
PlayerMovementController	Aceleración, fricción, salto y roll dive.
PlayerLadderController	Movimiento por escaleras.
PlayerCombatController	Ventana de ataque, hitbox e impactos.
PlayerLifecycleController	Muerte, reinicio y posición inicial.
PlayerPortalController	Integración con portales.
CharacterPhysicsController	Gravedad, damping y knockback.

7. Combate y sistema de vida

El ataque cuerpo a cuerpo activa una hitbox durante frames específicos de la animación. La configuración del jugador usa frames 2 a 3, daño 20 y retroceso.

7.1 Secuencia del ataque

1. Pulsa X mientras estás en el suelo.
2. El controlador entra en estado ATAQUE y reproduce la animación.
3. La hitbox se activa durante la ventana de frames configurada.
4. CombatHitProcessor procesa las áreas de daño del enemigo.
5. Al terminar la animación, el jugador vuelve al estado normal.

7.2 Vida y daño

Danable es la clase base para entidades con vida. Emite vida_cambiada y murio, y define funciones para morir, reiniciarse, recibir empuje y aplicar efectos. El HUD escucha vida_cambiada. El proyecto no declara un contador global de vidas; la derrota corresponde al jugador del nivel.

7.3 Enemigos

El enemigo incluye estados de patrulla, persecución, anticipación, ataque y recuperación, además de detección del jugador y navegación por plataformas. Danable filtra daño entre miembros de una misma facción.

8. Armas y aparición en el escenario

Hay diez escenas de armas: Basuca, Desert, Escopeta, Hacha, M4A1, Machete, Miniusi, Sable, Snyper y USP. scripts/SpawnArmas.gd recoge Marker2D hijos y selecciona puntos para cada lote.

Parámetro	Inicial	Uso
armas_escenas	10 escenas	Catálogo de pickups.
tiempo_respawn	15 s	Intervalo de renovación.
cantidad_activa	3	Puntos por lote.
estrategia_puntos	RandomSpawnPointStrategy	Orden aleatorio de puntos.

Al iniciar se genera un lote; al vencer el temporizador se eliminan pickups que queden y se crea otro. El script scripts/arma.gd detecta al jugador y elimina el pickup; su comentario deja la integración con inventario/equipamiento como trabajo pendiente.

8.1 Añadir un arma

1. Crear una escena siguiendo la estructura de las armas existentes.
2. Añadir el PackedScene al arreglo armas_escenas.
3. Verificar que el nivel tenga suficientes Marker2D.
4. Ejecutar y probar aparición, recogida y renovación.

9. Enemigos y progresión

EnemySpawner crea una oleada desde sus Marker2D. Su configuración predeterminada declara cinco enemigos iniciales, máximo diez y aparición automática desactivada; cada escena puede sobrescribir valores desde el Inspector.

9.1 Flujo de la oleada

```
Marker2D -> estrategia elige ubicación
EnemyFactory -> instancia enemy.tscn
enemy_spawned -> HUD agrega barra
todos derrotados -> vista de victoria
Enter -> carga nivel siguiente
```

Elemento	Responsabilidad
EnemyFactory	Instancia una escena de enemigo.
RandomSpawnPointStrategy	Mezcla puntos y devuelve la cantidad solicitada.
enemy_spawned	Notifica al HUD de una nueva instancia.
enemy_removed	Marca la barra como derrotada.
todos_los_enemigos_derrotados	Activa el flujo de victoria.

Las cantidades efectivas dependen del Inspector de cada escena; revisar también EnemySpawner al cambiar el diseño de oleadas.

10. Plataformas, escaleras y portales

Objeto	Interacción
Plataforma móvil	AnimatableBody2D; el jugador hereda velocidad al salir.
Escalera	Conecta alturas y plataformas.
Portal	Envía jugador/enemigo a otro portal con SpawnPoint; bloquea reentrada temporal por cuerpo.
Agua / damping	scripts/agua.gd y propiedades de damping alteran la física.

10.1 Configurar portales

- Asigna otro_portal en el Inspector.
- El destino debe tener un hijo Node2D llamado SpawnPoint.
- El bloqueo individual previene rebotes inmediatos entre portales.
- Los viajeros válidos pertenecen a Player, player o enemy.

11. HUD e interfaz

hub.tscn es CanvasLayer con barra de vida del jugador y contenedor de barras enemigas. Hub conecta las señales de vida y del spawner para mantener la interfaz actualizada.

1. Al iniciar, establece la barra del jugador con vida_actual y vida_maxima.
2. vida_cambiada actualiza el valor de esa barra.
3. enemy_spawned agrega fila y nombre al enemigo.
4. enemy_removed marca su barra como derrotada.
5. enemies_reset elimina las filas de la oleada anterior.

LevelMessageView muestra victoria y derrota. Enter continúa después de la victoria; R solicita reinicio tras la derrota.

12. Arquitectura y patrones

Patrón	Aplicación	Aporte
Observer / señales	Vida, spawns, victoria y muerte.	Comunicación reactiva entre nodos.
Factory	WeaponFactory, EnemyFactory.	Instanciación común de PackedScene.
Strategy	Selección aleatoria de puntos.	Cambia algoritmo sin reescribir spawner.
Componentes	Controladores en scripts/components.	Divide responsabilidades.
Autoload	MusicManager.	Servicio de música entre escenas.
Facade	Métodos del gestor y coordinadores.	Interfaz simple para operaciones.

Al ampliar el juego, conserva cada responsabilidad en su controlador y usa señales para comunicar cambios entre sistemas.

13. Reinicio, victoria y continuidad

Situación	Resultado
Muerte del jugador	Se cancela victoria pendiente y aparece derrota.
R tras derrota	Oculta mensaje, reinicia jugador y oleada.
Oleada derrotada	Muestra victoria; puede esperar antes de transición.
Enter tras victoria	Carga escena_siguiente si no hay transición automática.

14. Niveles y escena final

Escena	Destino	Estado
nivel1.tscn	nivel2.tscn	Nivel inicial.
nivel2.tscn	nivel3.tscn	Nivel intermedio.
nivel3.tscn	escena_final_rescate.tscn	Destino es un marcador de posición.

Las referencias PackedScene confirman el orden. Los mapas se editan gráficamente y los valores de enemigos dependen del Inspector.

Al terminar nivel 3 se cambia de escena, pero el rescate no está terminado. Hace falta diseñar la pantalla y decidir cómo volver al menú o cerrar la experiencia.

15. Audio y recursos visuales

MusicManager es autoload y carga assets/audio/musica_niveles.ogg. Juego solicita reproducir música al iniciar el nivel. El repositorio también contiene efectos para salto, impactos, armas, recogida y menú; su uso depende de las conexiones de cada escena.

Los sprites están agrupados por acción: quieto, caminar, correr, salto, agachado, roll dive, ataque, daño y caída. Los escenarios viven en assets/sprites/escenarios y el audio en assets/audio.

Conserva licencias y atribuciones de fuentes, sprites, audio y complementos. Live Palette declara MIT en addons/live_palette/LICENSE.

16. Pruebas recomendadas

1. Ejecutar el menú con F5 y pulsar Jugar.
2. Probar movimiento, correr, salto, agachado, ataque y roll dive.
3. Subir y bajar una escalera.
4. Usar portales y verificar SpawnPoint.
5. Esperar aparición de armas y recoger un pickup.
6. Recibir y causar daño; comprobar barras y animaciones.
7. Derrotar toda la oleada y avanzar con Enter.
8. Repetir en los tres niveles; confirmar que nivel 3 abre la escena final pendiente.
9. Morir y pulsar R; comprobar reinicio de jugador y oleada.

17. Errores frecuentes y diagnóstico

Problema	Causa probable	Revisar
No aparece oleada	Faltan Marker2D o spawner sin configurar.	Árbol, cantidad inicial, warnings.
HUD vacío	Jugador/spawner sin asignar.	Exports de hub.tscn.
No avanza nivel	escena_siguiente vacía o quedan enemigos.	Export de Juego y señal de victoria.
Portal no funciona	otro_portal sin asignar o sin SpawnPoint.	Referencias de escena destino.
R no reinicia	Mensaje no visible o input no recibido.	LevelMessageView y _unhandled_input.
No salen armas	Pocos puntos o catálogo vacío.	cantidad_activa y armas_escenas.
Falta música	Autoload o reproductor no activo.	MusicManager y llamada reproducir_musica.

Depuración

Ejecuta desde el editor y revisa Output. Los warnings indican puntos o referencias faltantes. El Remote Scene Tree permite confirmar qué instancias están vivas.

18. Actividades de aprendizaje

18.1 Análisis

- Explica cómo la señal de oleada derrotada muestra victoria.
- Describe qué responsabilidades separa player.gd en controladores.
- ¿Qué ocurre si cantidad_activa supera los Marker2D disponibles?
- ¿Cómo evita el bloqueo individual el rebote entre portales?

18.2 Modificación

- Ajusta tiempo_respawn y compara frecuencia de pickups.
- Añade Marker2D y observa la selección aleatoria.
- Cambia cantidad inicial de enemigos y valida las barras del HUD.
- Agrega una escena de arma al catálogo.
- Diseña una escena final de rescate y define cómo regresar al menú.

19. Lista de verificación

Criterio	OK
Proyecto abre desde project.godot.	□
Menú inicia y cierra el juego.	□
Controles principales responden.	□
Jugador recibe daño y se reinicia.	□
HUD muestra vida del jugador y enemigos.	□
Enemigos aparecen desde sus puntos.	□
Armas aparecen y se recogen.	□
Portales usan SpawnPoint válido.	□
Oleada derrotada muestra victoria.	□
Enter carga el nivel siguiente.	□
Nivel 3 abre escena final.	□
Escena final está completa antes de publicar.	□

20. Referencias y créditos

Fuente técnica: proyecto Megafighters, especialmente project.godot, scenes/*.tscn y scripts/*.gd. Motor: Godot Engine 4.7 según su configuración. Documentación oficial: docs.godotengine.org. Respeta las licencias incluidas junto a tipografías, sprites, audio y complementos.

21. Próximas mejoras

El pendiente más visible es completar la escena de rescate. Antes de publicar, conviene terminar esa pantalla y enlazarla con una salida clara. Como ampliaciones posteriores, documentar cómo se equipa cada arma, validar el comportamiento jugable de todos los pickups y definir balance de daño y vida.

Fin del manual · Megafighters